

Flink YARN 模块

核心流程与设计说明

基于 Apache Flink 1.15.4 Release 分支源码分析

flink-yarn 模块完整解读

模块路径: `flink-yarn/src/main/java/org/apache/flink/yarn/`

目录

- 一、模块概览与架构设计
- 二、核心类设计说明
- 三、集群部署核心流程
- 四、资源管理核心流程
- 五、Session vs Application 模式
- 六、文件上传与资源注册
- 七、CLI 命令行交互
- 八、配置体系
- 九、安全机制
- 十、关键设计模式

一、模块概览与架构设计

1.1 模块定位

flink-yarn 模块是 Apache Flink 与 Hadoop YARN 集成的核心桥梁。负责将 Flink 集群部署到 YARN，管理 YARN 容器中 TaskExecutor 进程的生命周期，处理所有 YARN 特有的资源分配、文件分发和安全认证。

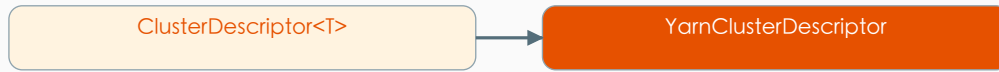
1.2 模块包结构

包名	职责	核心类数
o.a.f.yarn	核心实现: 集群描述、资源管理、工具	15+
o.a.f.yarn.cli	CLI 命令行交互	4
o.a.f.yarn.configuration	YARN 配置项	5
o.a.f.yarn.entrypoint	AM 入口点	5
o.a.f.yarn.executors	执行器工厂	2

1.3 核心类层次结构

核心类层次结构

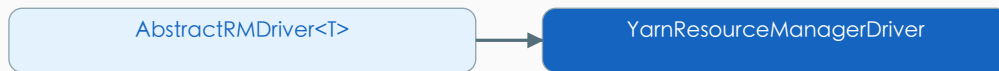
集群部署层



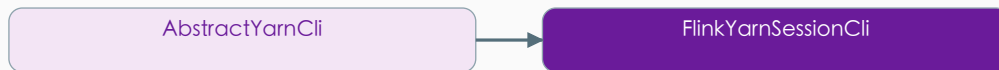
入口点层



资源管理层



CLI层



工具适配层



图 1-1: 核心类层次结构

二、核心类设计说明

2.1 YarnClusterDescriptor (约1900行)

实现 ClusterDescriptor<ApplicationId> 接口，是集群部署的核心协调者。

- deploySessionCluster()/deployApplicationCluster(): 两种部署模式统一入口，内部调用 deployInternal()
- isReadyForDeployment(): 检查 flinkJarPath、YARN vCores 上限、Hadoop 环境变量
- validateClusterResources():
内存对齐(yarn.scheduler.minimum-allocation-mb)、最大资源限制
- startAppMaster(): 核心方法，上传文件、构建AM容器、提交到YARN并轮询等待RUNNING
- setupApplicationMasterContainer(): 通过模板替换构建JVM启动命令
- DeploymentFailureHook: 内部Thread子类，注册ShutdownHook防止部署失败时资源泄漏
- ApplicationSubmissionContextReflector: 反射兼容不同Hadoop版本(2.4.0+/2.6.0+)
- setTokensFor(): Kerberos delegation token 注入到容器上下文

2.2 YarnResourceManagerDriver (约880行)

继承

AbstractResourceManagerDriver<YarnWorkerNode>，运行在AM进程中，负责动态容器管理。

- 双客户端架构: AMRMClientAsync(与YARN RM通信) + NMClientAsync(与NodeManager通信)
- 心跳自适应: 有容器请求时快心跳，无请求时慢心跳，减少YARN RM负担
- requestResourceFutures: Map> 异步资源请求队列
- AMRMCallbackHandler: 处理
onContainersAllocated/onContainersCompleted/onShutdownRequest
- NMCallbackHandler: 处理容器启动/停止回调，使用Phaser跟踪释放进度
- getContainersFromPreviousAttempts(): AM 容器恢复(HA场景)
- tryUpdateApplicationBlockList(): 动态节点黑名单管理

2.3 YarnApplicationFileUploader (约560行)

负责文件上传到HDFS(.flink/<appld>/)并注册为YARN LocalResource，实现AutoCloseable。

- registerSingleLocalResource(): 上传单文件，支持APPLICATION/PUBLIC可见性
- registerMultipleLocalResources(): 递归处理目录，排除flink-dist*.jar，构建classpath
- uploadFlinkDist(): 单独处理Flink发行版JAR，确保不重复
- registerProvidedLocalResources(): PUBLIC可见性的共享库，支持跨应用YARN缓存
- waitForTransferToComplete(): 处理HDFS最终一致性(重试3次)

2.4 FlinkYarnSessionCli (约977行)

YARN Session 模式命令行入口，继承 AbstractYarnCli。

- 解析参数: -jm/-tm(内存), -s(slots), -qu(队列), -nm(名称), -D(动态属性)
- YARN Properties 文件: .yarn-properties-{user} 支持跨进程会话恢复
- runInteractiveCli(): 交互式CLI, help/stop命令, 持续监控应用状态
- run(): 主流程 — 创建描述符 -> 部署/检索集群 -> 写属性文件 -> 交互/detach

2.5 Utils 工具类 (约823行)

- createTaskExecutorContext():
构建TM容器完整上下文(LocalResource+环境变量+安全令牌)
- getTaskManagerShellCommand(): 模板替换生成JVM启动命令
- setupYarnClassPath(): 配置YARN类路径
- getYarnAndHadoopConfiguration(): 合并Flink和Hadoop配置
- setAclsFor(): 设置YARN应用ACL(VIEW_APP/MODIFY_APP)

2.6 其他核心类

类名	职责
YarnWorkerNode	封装YARN Container和ResourceID, 代表一个TaskExecutor
TaskExecutorProcessSpec ContainerResourcePriority Adapter	Flink TaskExecutorProcessSpec 与 YARN Resource/Priority 的双向映射适配器
YarnConfigOptions	526行配置项定义: 队列、内存、端口、安全、文件分发等全部YARN配置
YarnDeploymentTarget	枚举: SESSION("yarn-session") / APPLICATION("yarn-application")
YarnResourceManagerFactory	单例工厂, 创建 YarnResourceManagerDriver, 不支持多Leader会话
YarnEntrypointUtils	AM入口工具: 加载配置、解析keytab路径、日志YARN环境信息
YarnLocalResourceDescriptor	YARN本地资源描述符: 封装路径/大小/时间戳/可见性/类型, 支持序列化

三、集群部署核心流程

集群部署是 `flink-yarn` 模块最核心的流程，由 `YarnClusterDescriptor.deployInternal()` 方法驱动。完整流程包括 Kerberos 安全检查、YARN 资源验证、文件上传、AM 容器构建和应用提交。



图 3-1: 集群部署核心流程

关键实现细节:

- `deployInternal()` 先检查 Kerberos 凭据有效性，若启用安全但凭据无效则抛出异常
- YARN 资源验证包括: 内存对齐到 `yarn.scheduler.minimum-allocation-mb` 的整数倍、不超过集群最大容器资源
- `startAppMaster()` 是最长的方法(约480行): 初始化文件系统 -> 创建FileUploader -> 上传系统/用户文件 -> 构建classpath -> 写JobGraph/配置 -> 处理Kerberos -> 创建ContainerLaunchContext -> 提交 -> 轮询状态
- 提交后进入轮询循环(250ms间隔): NEW -> SUBMITTED -> ACCEPTED -> RUNNING (成功) 或 FAILED/KILLED (失败)
- `DeploymentFailureHook` 在轮询过程中保护资源，部署成功后移除

四、资源管理核心流程

资源管理流程运行在 AM 进程内部，由 YarnResourceManagerDriver 驱动，通过 YARN 的异步客户端与 YARN RM 和 NodeManager 交互。

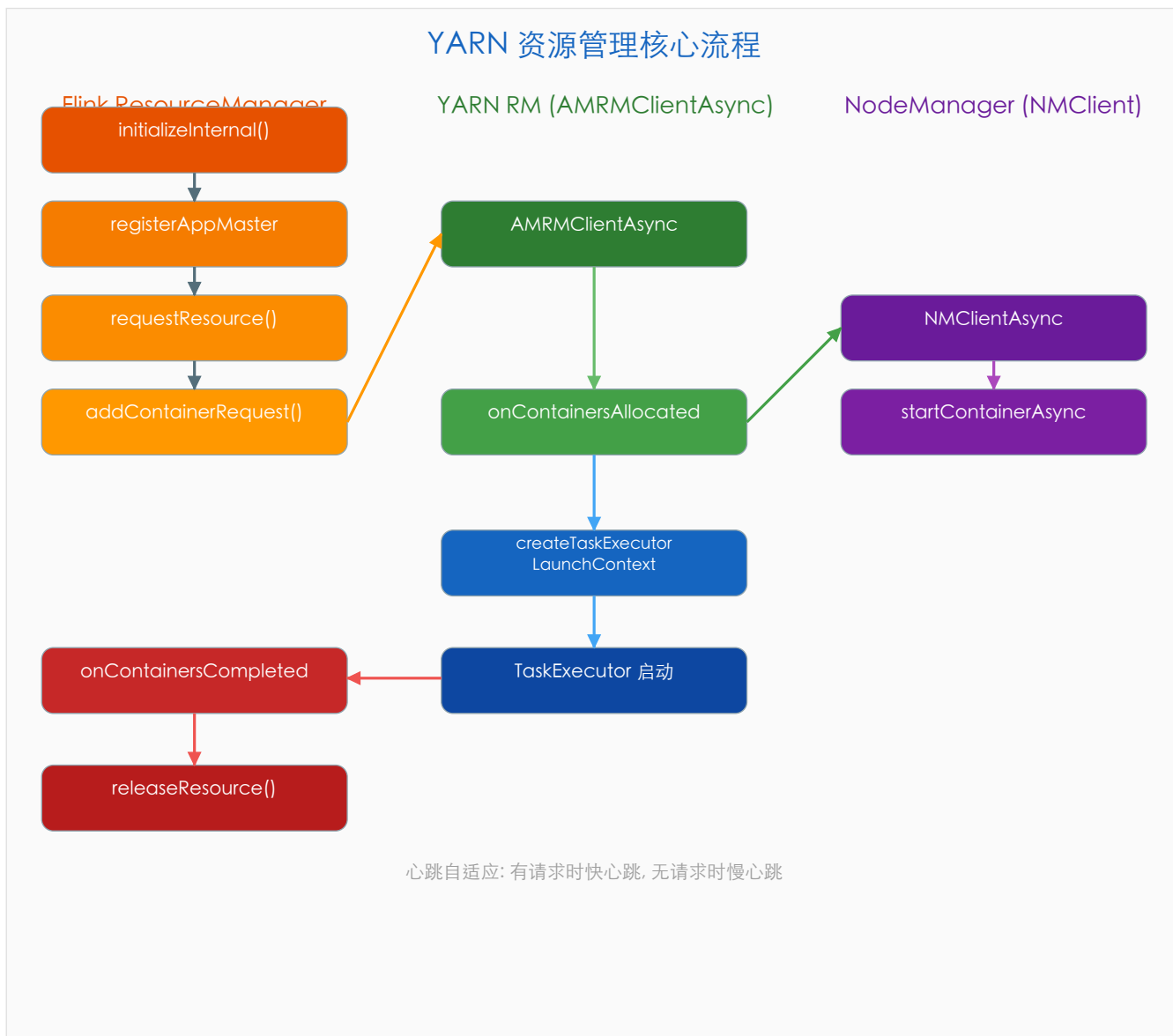


图 4-1: 资源管理核心流程

核心交互流程:

- 初始化: 创建并启动 AMRMClientAsync 和 NMClientAsync，向YARN RM注册AM
- 资源请求: Flink SlotManager 通过 requestResource() 发起，转化为 YARN ContainerRequest
- TaskExecutorProcessSpecContainerResourcePriorityAdapter 将 Flink 的 TaskExecutorProcessSpec 转为 YARN Resource+Priority
- 容器分配回调: onContainersAllocated() 按Priority分组处理，匹配到 requestResourceFutures 队列

- 容器启动: `createTaskExecutorLaunchContext()` 在IO线程池异步构建，通过 `NMClientAsync` 启动
- 容器完成: `onContainersCompleted()` 通知 `ResourceEventHandler`，处理不同退出状态(PREEMPTED/ABORTED/DISKS_FAILED等)
- 容器释放: `releaseResource()` 同时通知 NM 停止容器和通知 RM 释放分配
- 优雅终止: `terminate()` 使用 `Phaser` 等待所有容器停止后再关闭客户端

五、Session 模式 vs Application 模式

Flink on YARN 支持两种部署模式，通过 YarnDeploymentTarget 枚举区分。两种模式在入口点、Job图生成位置和资源隔离性上有本质差异。

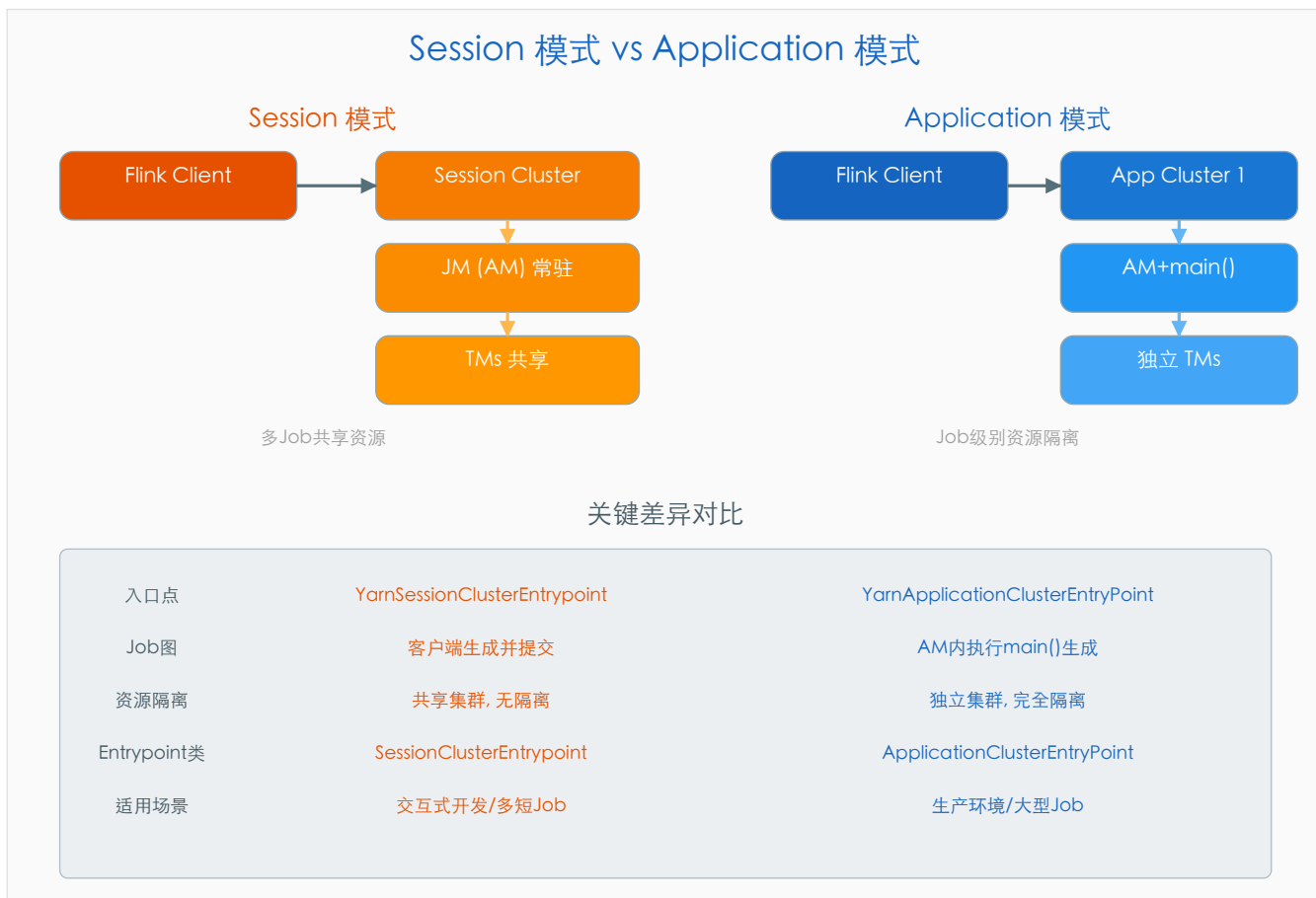


图 5-1: Session 模式 vs Application 模式对比

Session 模式详解:

- 入口点: YarnSessionClusterEntrypoint -> SessionClusterEntrypoint
- 部署: deploySessionCluster() -> deployInternal(), AM启动后等待Job提交
- Job图由客户端生成并通过REST API提交到已运行的Session集群
- 多个Job共享同一个JM和TM资源池, 适合交互式开发和短任务

Application 模式详解:

- 入口点: YarnApplicationClusterEntryPoint -> ApplicationClusterEntryPoint
- 部署: deployApplicationCluster(), 用户JAR被上传到AM, 在AM内执行main()生成Job图
- 每个Application独立拥有JM和TM, Job级别完全隔离
- AM内通过 DefaultPackagedProgramRetriever 获取 PackagedProgram 并执行

六、文件上传与资源注册

文件上传由 YarnApplicationFileUploader 管理，是部署流程中最复杂的环节之一。需要将本地的JAR包、配置文件、插件等上传到HDFS，并注册为YARN LocalResource。

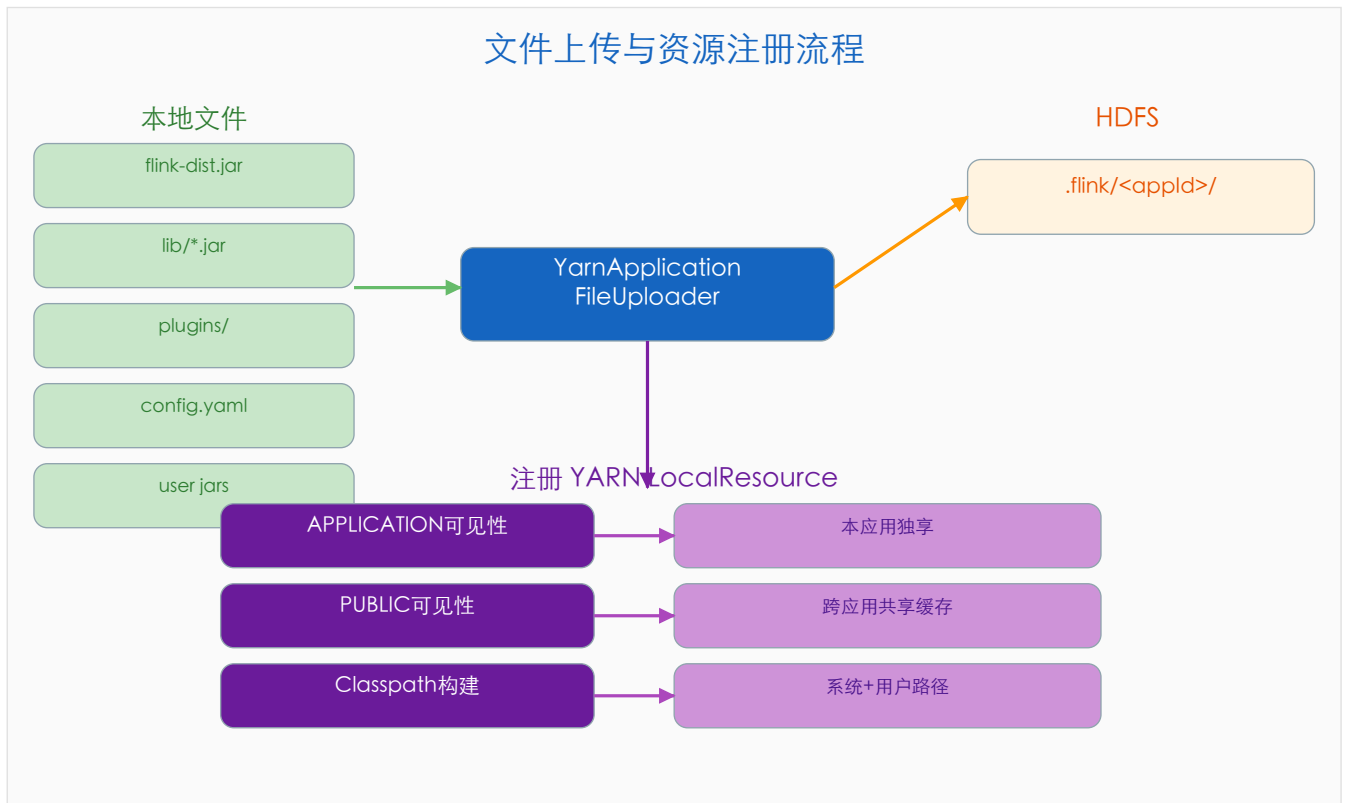


图 6-1: 文件上传与资源注册流程

Classpath 构建策略:

- UserJarInclusion.FIRST: 用户JAR排在系统JAR之前
- UserJarInclusion.ORDER: 用户JAR混入系统JAR一起排序
- UserJarInclusion.LAST: 用户JAR排在系统JAR之后(默认)
- UserJarInclusion.DISABLED: 用户JAR放入usrlib目录，由UserClassLoader加载

七、CLI 命令行交互

FlinkYarnSessionCli 提供了完整的 YARN Session 管理命令行界面。

参数	说明
-jm / --jobManagerMemory	JobManager 内存，支持单位(1024m, 1g)
-tm / --taskManagerMemory	TaskManager 内存
-s / --slots	每个 TaskManager 的 slot 数
-qu / --queue	YARN 队列名
-nm / --name	应用名称
-at / --applicationType	应用类型
-nl / --nodeLabel	YARN 节点标签
-z / --zookeeperNamespace	ZK 命名空间(HA模式)
-id / --applicationId	连接已有Session
-d / --detached	分离模式(后台运行)
-D property=value	动态配置属性

八、配置体系

YarnConfigOptions 定义了526行YARN专有配置，是模块配置的中枢。

配置分类	关键配置项	说明
资源配置	yarn.appmaster.vcores yarn.containers.vcores	AM/TM 虚拟核心数
队列配置	yarn.application.queue	YARN 队列
文件分发	yarn.ship-files yarn.ship-archives yarn.provided-lib-dirs	本地文件/归档/共享库
安全配置	yarn.ship-local-keytab yarn.security.kerberos.localized-keytab-path	Keytab 分发策略
AM端口	yarn.application-master.port	AM RPC 端口范围
心跳	yarn.heartbeat-delay yarn.container-request-heartbeat-interval-milliseconds	心跳间隔配置
高可用	yarn.application-attempts yarn.application-attempt-failures-validity-interval	HA 重试配置
日志	yarn.rolled-log-include-pattern yarn.rolled-log-exclude-pattern	日志聚合过滤
部署模式	execution.target	yarn-session / yarn-application

九、安全机制

flink-yarn 模块对 Kerberos 安全和 Delegation Token 有完善的支持:

- Keytab 分发: 支持 ship-local-keytab(上传到HDFS) 和 pre-installed(预装在节点上) 两种方式
- Delegation Token: 通过 DefaultDelegationTokenManager 获取 HDFS/Hive/HBase 等服务的 token
- AM Token 注入: setTokensFor() 将所有 token 序列化到 ContainerLaunchContext
- TM Token 传递: TaskExecutor 从 HADOOP_TOKEN_FILE_LOCATION 读取 token 文件, 过滤掉 AMRMTOKEN
- KRB5 配置: 支持将 krb5.conf 作为 LocalResource 分发到容器
- 安全模式检测: HadoopUtils.isKerberosSecurityEnabled() 检查是否启用 Kerberos
- SecurityUtils.runSecured(): CLI 在安全上下文中执行所有操作

十、关键设计模式与架构思想

工厂模式

YarnClusterClientFactory / YarnResourceManagerFactory /
DefaultYarnResourceManagerClientFactory — 所有核心组件通过工厂创建，解耦实例化逻辑

模板方法

AbstractResourceManagerDriver.initializeInternal() 由子类 YarnResourceManagerDriver 实现；AbstractYarnCli 定义 CLI 框架

策略模式

UserJarInclusion (FIRST/ORDER/LAST/DISABLED) 控制classpath构建策略

回调模式

AMRMCallbackHandler / NMCallbackHandler — YARN 异步客户端的事件处理

反射适配

ApplicationSubmissionContextReflector / ContainerRequestReflector /
ResourceInformationReflector — 兼容不同Hadoop版本

单例模式

YarnResourceManagerFactory / AMRMClientAsyncReflector /
ContainerRequestReflector / ResourceInformationReflector

CompletableFuture异步编排

requestResource() 返回 Future，容器分配后
complete()，支持取消(CancellationException)

ShutdownHook保护

DeploymentFailureHook 防止部署失败时资源泄漏；FlinkYarnSessionCli 关闭时清理集群

Phaser并发协调

trackerOfReleasedResources 使用 Phaser 而非 CountdownLatch，支持动态注册/注销

SPI 服务发现

YarnSessionClusterExecutorFactory 通过 META-INF/services 注册，实现插件化部署目标

文档结束 — 基于 Flink 1.15.4 flink-yarn 模块全部源码分析生成